



WS4 – Dynamic Simulations of Neuromechanics

WS4: Dynamic Simulations of Neuromechanics

DAY 1

From neuron to muscle excitation

- Intro to modelling (10m)
- LIF neuron model (10 m)
- From spikes to muscle excitation exercises (1h)
- Q&A

DAY 2

Forward Sim + Control

- **MuJoCo and forward modelling basics** (25 m)
- **Control basics and introducing programming challenge** (15 m)
- **Arm prototyping exercise** (1h)
- Q&A

DAY 3

Advanced use-cases

- **Reinforcement learning & motion synthesis** (10 m)
- **Simulations for vision-based robotics** (15 m)
- **Virtual prostheses, sim2real concerns and methods** (20 m)
- **Ankle exo challenge** (1h)
- Q&A + buffer

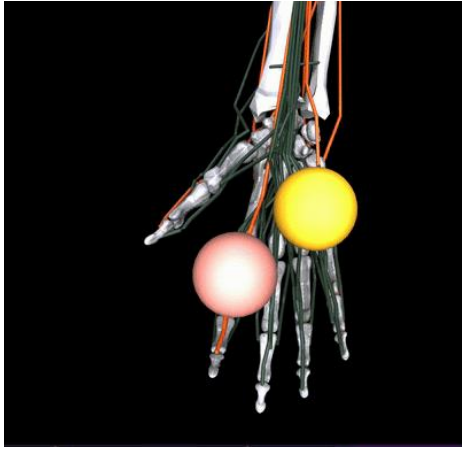
Why physics-based simulation?



Every interaction needs to be set up to not violate our expectations.

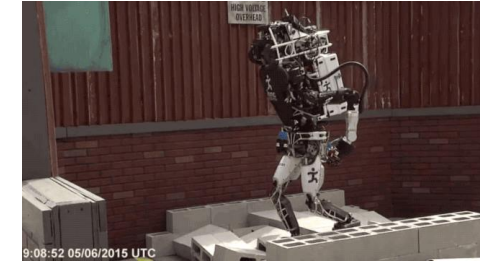
Can we enforce them implicitly?

Quick history of MuJoCo



MyoSuite

Optimal
control in
biomechanics



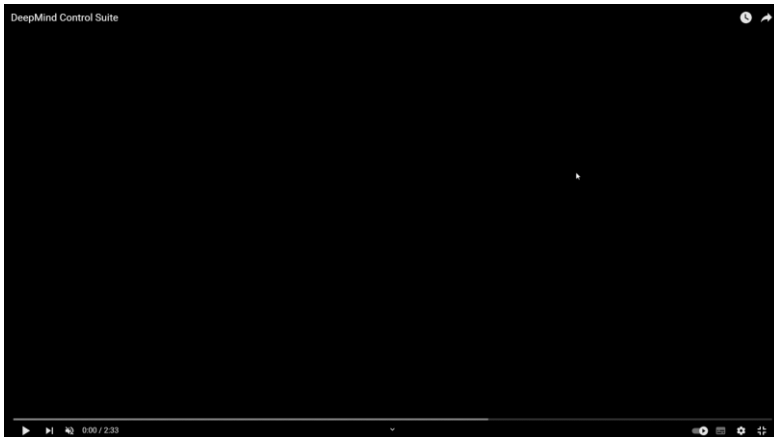
DARPA Robotics challenge

DeepMind
Acquisition +
Open Sourcing

Robotics
Community



RL Community



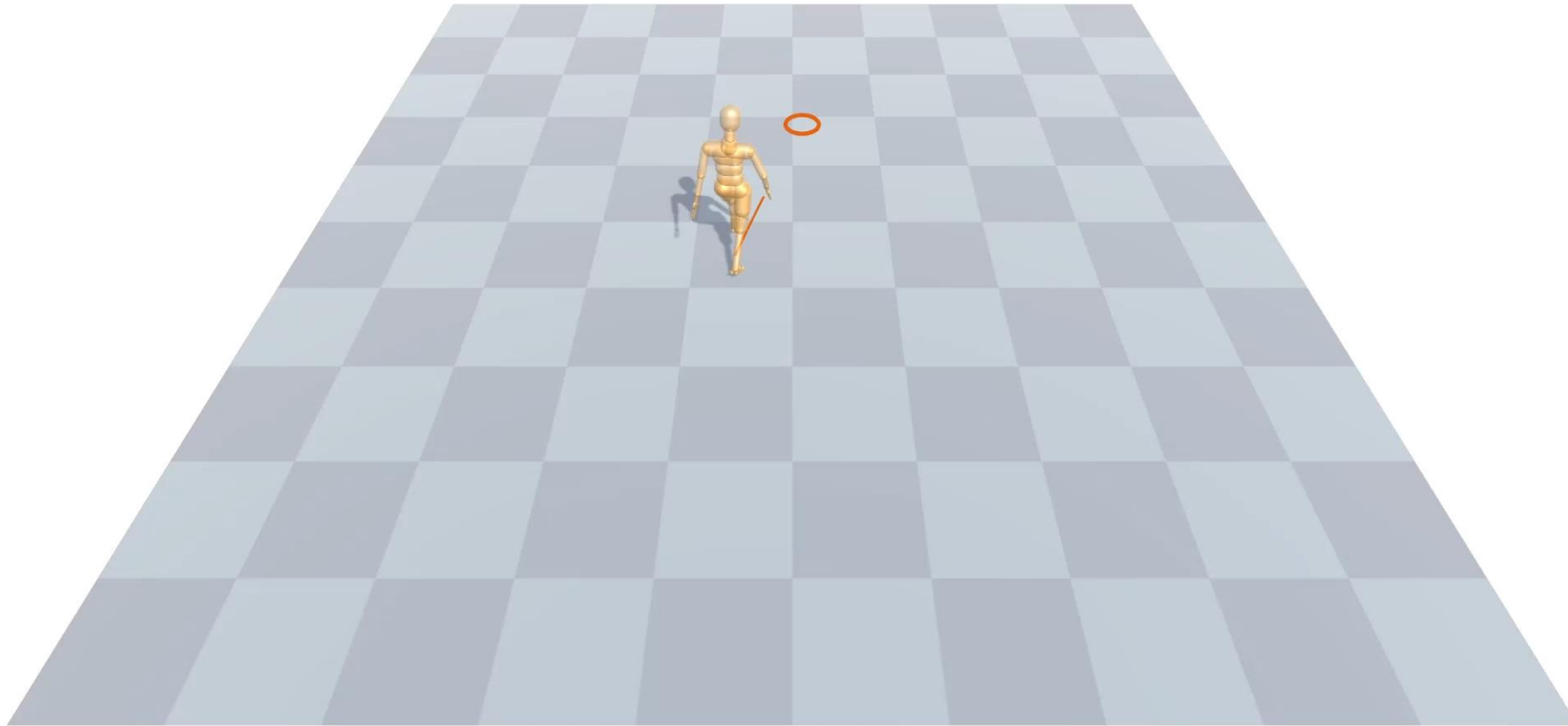
Example use case: Virtual Prototyping

- Same motor decode and motion capture
- Contact on virtual hand's middle finger activates force-graded stimulation of subject's phantom middle finger
- Higher force on middle finger causes a higher frequency of stimulation

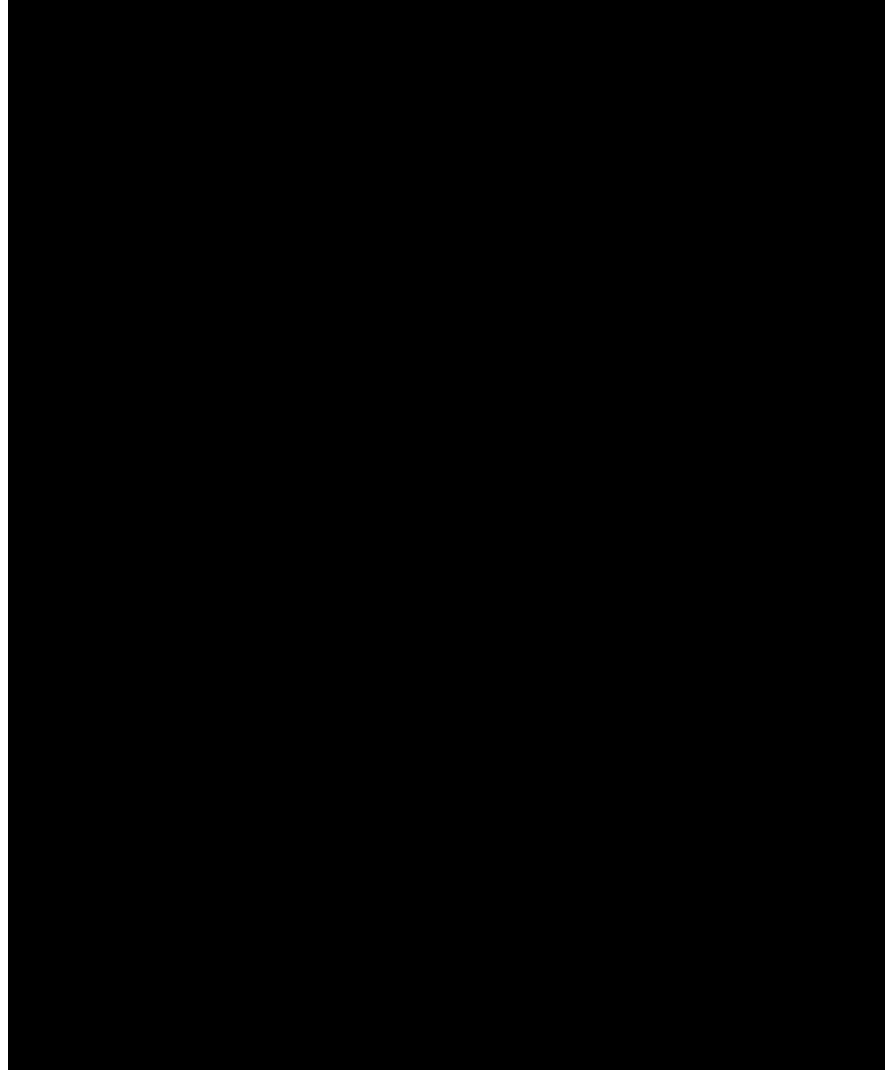
D. T. Kluger et al., "Virtual Reality Provides an Effective Platform for Functional Evaluations of Closed-Loop Neuromyoelectric Control," in IEEE TNSRE 2019

00:19 / 00:54

Example use case: Motion Synthesis



Example use case: Digital Twin



Example use case: Shared autonomy control with vision based control



Automatically controlled:

- Hand Preshape
- Hand Aperture
- Wrist Rotation

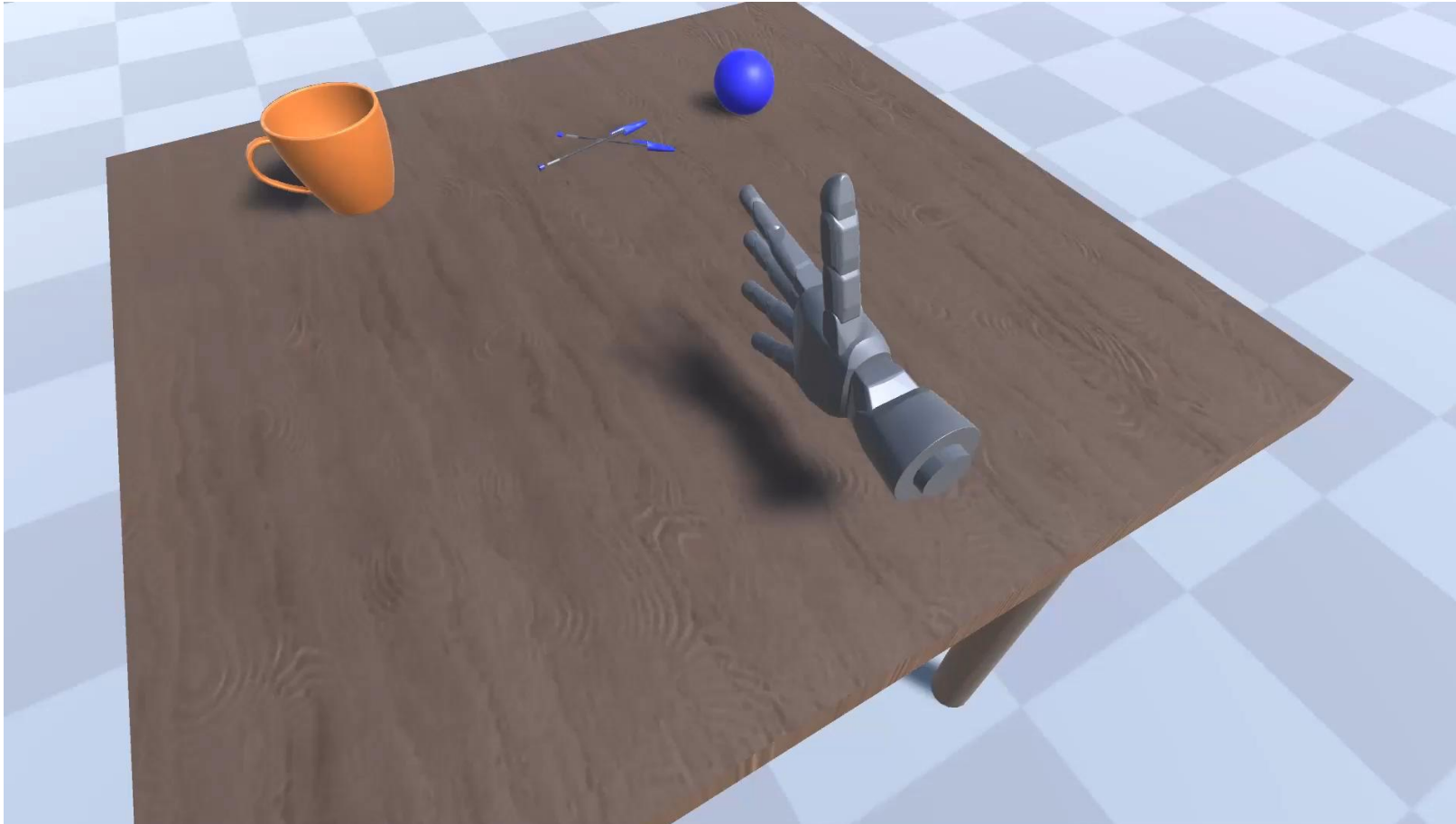
Myoelectrically controlled:

- Grasping/releasing

Reduce:

- Compensatory movement
- Muscle activation

Example use case: Generate datasets or evaluate performance in simulation



Example use case: Generate datasets or evaluate performance in simulation

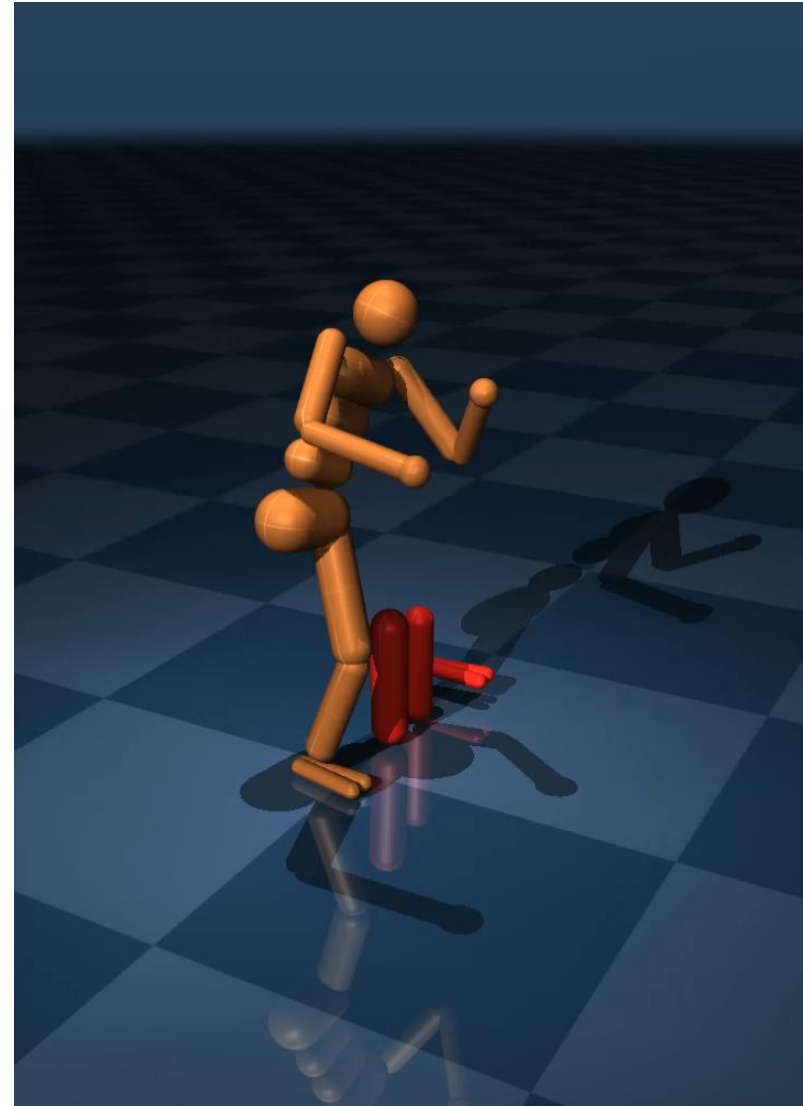


Developing with MuJoCo

- **Python, Simulink, C++, C# API / wrappers**
- **Visual GUI in Coppelia Sim or in Unity**
- **Custom GUIs**
- **GPU accelerated learning, engine rewritten in Jax (MJX)**

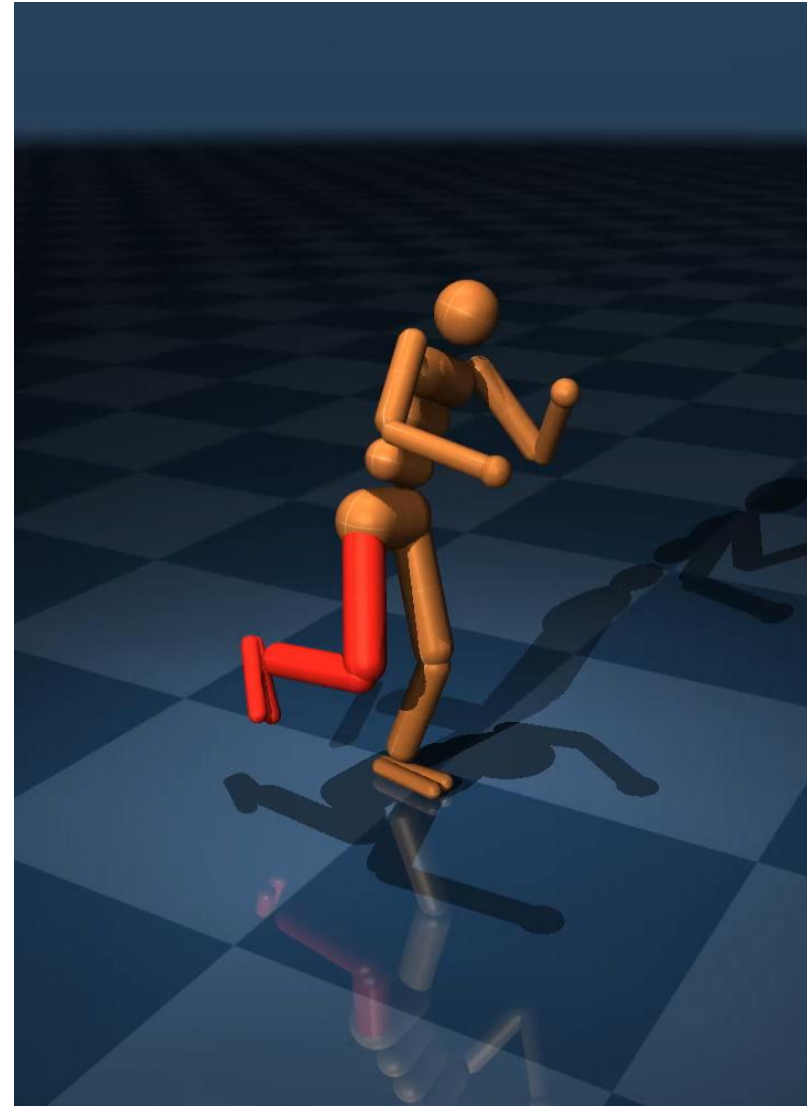
Simulation State

Global Coordinates



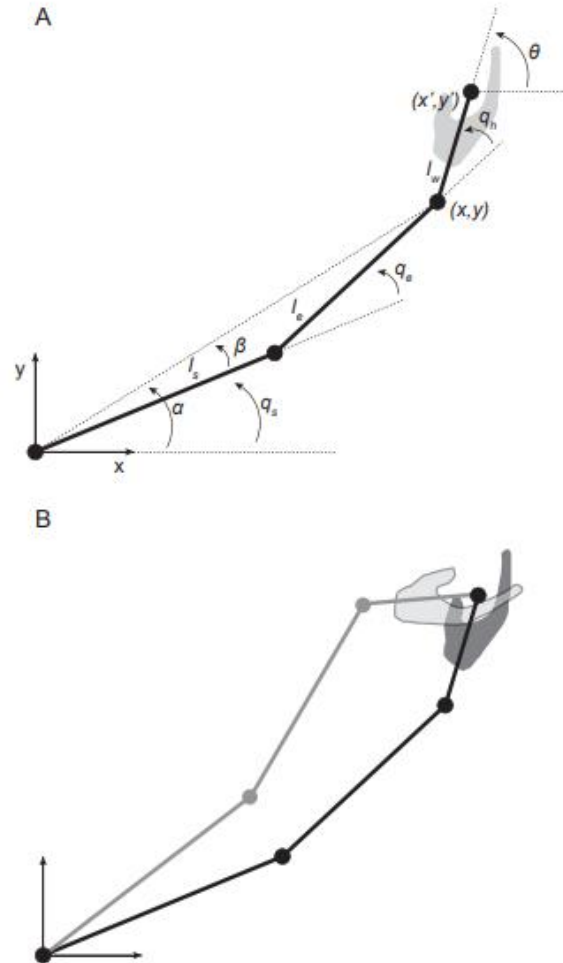
Simulation State

Generalised Coordinates



Simulation State

Not always 1:1 mapping between different state representations



Simulation State

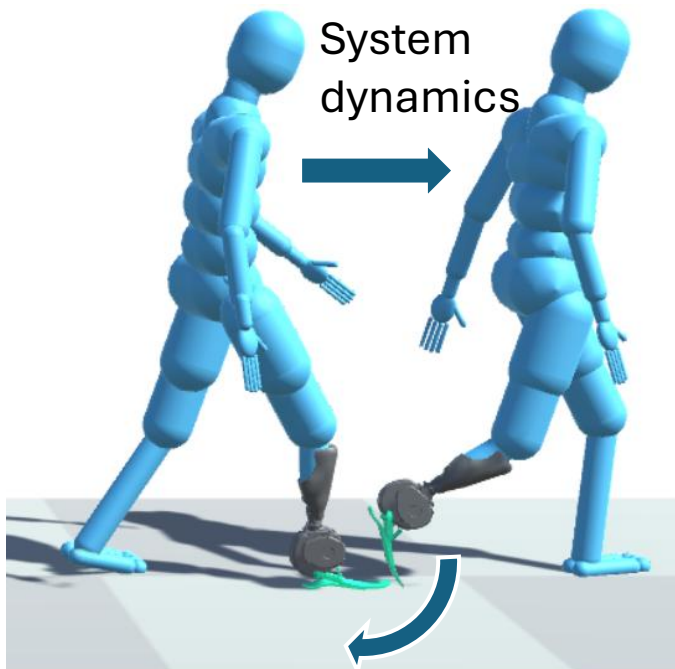
Convert between state representations

$$\mathbf{J}(\mathbf{q}) = \left(\frac{\partial x_i}{\partial q_j} \right) \longrightarrow \partial x = \mathbf{J} \partial q$$

Hand		Joint		Muscle	
$\mathbf{x}$	←	$\mathbf{q}$	→	λ	position
$\dot{\mathbf{x}}$	← $\mathbf{J}$	$\dot{\mathbf{q}}$	→ $\mathbf{J}_\mu$	$\dot{\lambda}$	velocity
$\mathbf{F}$	→ $\mathbf{J}^T$	$\boldsymbol{\tau}$	← $\mathbf{J}_\mu^T$	μ	force
$\mathbf{K}_x$	→ $\mathbf{J}^T \mathbf{K}_x \mathbf{J} + \frac{d\mathbf{J}^T}{dq} \mathbf{F}$	$\mathbf{K}$	← $\mathbf{J}_\mu^T \mathbf{K}_\mu \mathbf{J}_\mu + \frac{d\mathbf{J}_\mu^T}{dq} \mu$	$\mathbf{K}_\mu$	stiffness
$\mathbf{D}_x$	→ $\mathbf{J}^T \mathbf{D}_x \mathbf{J}$	$\mathbf{D}$	← $\mathbf{J}_\mu^T \mathbf{D}_\mu \mathbf{J}_\mu$	$\mathbf{D}_\mu$	damping

Forward vs. Inverse modelling

Forward



Applied forces and torques →
Observe resulting movement

Inverse



Observe movement →
Find forces/torques that explain it

Evolving State- Integration based simulation

Equation of motion

$$M\dot{v} = \sum \tau$$

$$M\dot{v} + c = \tau + J^T f$$

Forward modelling:

τ known, what is $\dot{v}$?

$$\dot{v} = M^{-1}(\tau + J^T f - c)$$

Inverse modelling:

$\dot{v}$ known, what is τ ?

$$\tau = M\dot{v} + c - J^T f$$

Symbol	Size	Description
n_Q		number of position coordinates
n_V		number of degrees of freedom
n_C		number of active constraints
q	n_Q	joint position
v	n_V	joint velocity
τ	n_V	applied force: passive, actuation, external
$c(q, v)$	n_V	bias force: Coriolis, centrifugal, gravitational
$M(q)$	$n_V \times n_V$	inertia in joint space
$J(q)$	$n_C \times n_V$	constraint Jacobian
$r(q)$	n_C	constraint residual
$f(q, v, \tau)$	n_C	constraint force

Evolving State- Integration based simulation

Timestep of h seconds (e.g. 0.005 s)

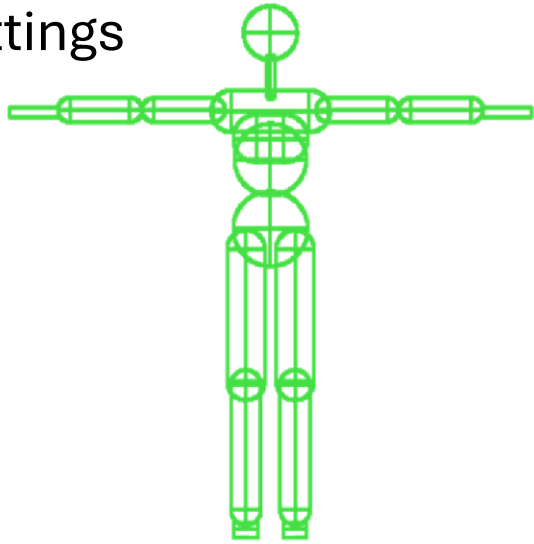
velocity: $v_{t+h} = v_t + ha_t$

position: $q_{t+h} = q_t + hv_t$

System representation in MuJoCo

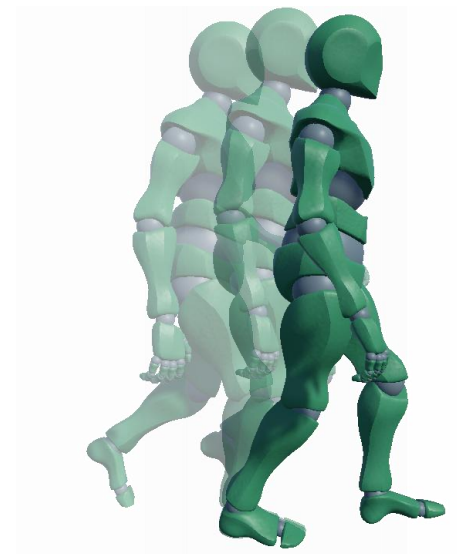
Model

- Static information about model
- Reference frames, DoFs
- Inertia, geometry, colliders, constraints
- Passive properties, actuation setup
- Sim settings



Data

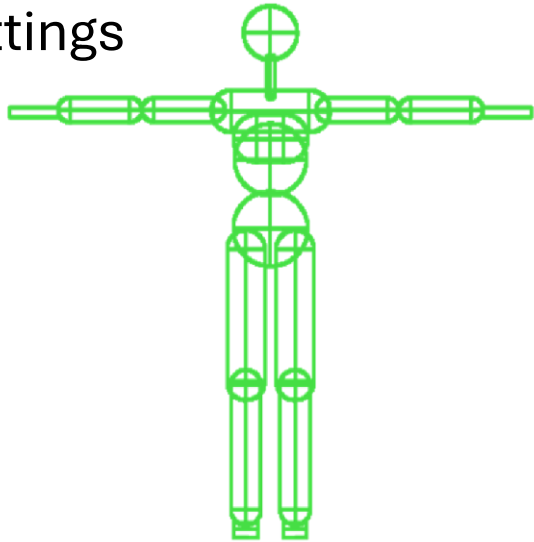
- Dynamic information about the model
- Current pose and derivatives
- Jacobians, mass matrix, contact state
- Actuation and control logic



System representation in MuJoCo

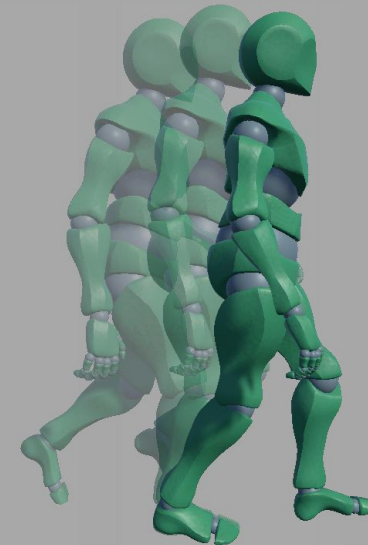
Model

- Static information about model
- Reference frames, DoFs
- Inertia, geometry, colliders, constraints
- Passive properties, actuation setup
- Sim settings



Data

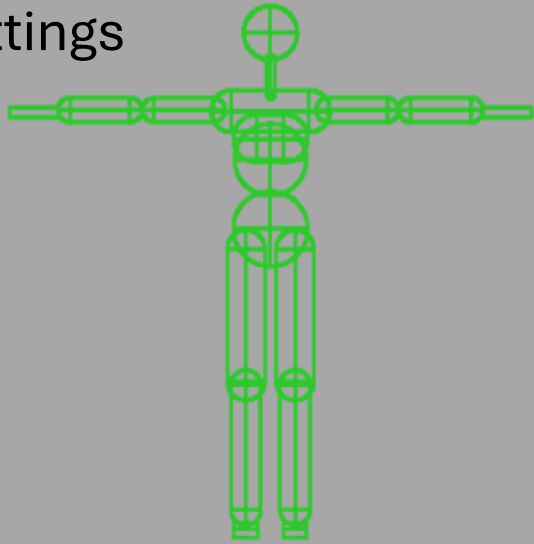
- Dynamic information about the model
- Current pose and derivatives
- Jacobians, mass matrix, contact state
- Actuation and control logic



System representation in MuJoCo

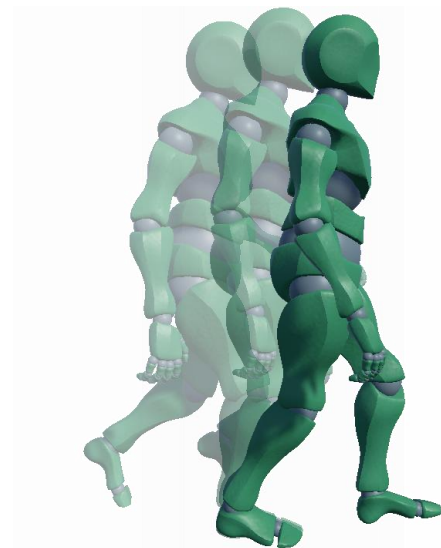
Model

- Static information about model
- Reference frames, DoFs
- Inertia, geometry, colliders, constraints
- Passive properties, actuation setup
- Sim settings



Data

- Dynamic information about the model
- Current pose and derivatives
- Jacobians, mass matrix, contact state
- Actuation and control logic



Simulation Loop

```
void mj_step1(const mjModel* m, mjData* d)
{
    mj_checkPos(m, d);
    mj_checkVel(m, d);
    mj_fwdPosition(m, d);
    mj_sensorPos(m, d);
    mj_energyPos(m, d);
    mj_fwdVelocity(m, d);
    mj_sensorVel(m, d);
    mj_energyVel(m, d);

    // if we had a callback we would be using mj_step, but call it anyway
    if( mjcb_control )
        mjcb_control(m, d);
}

void mj_step2(const mjModel* m, mjData* d)
{
    mj_fwdActuation(m, d);
    mj_fwdAcceleration(m, d);
    mj_fwdConstraint(m, d);
    mj_sensorAcc(m, d);
    mj_checkAcc(m, d);

    // compare forward and inverse solutions if enabled
    if( mjENABLED(mjENBL_FWDINV) )
        mj_compareFwdInv(m, d);

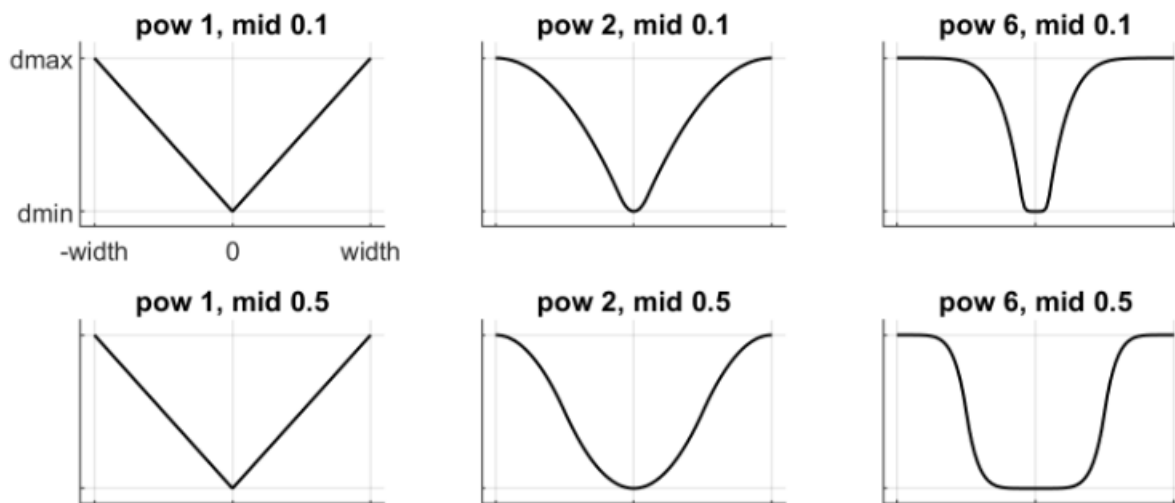
    // integrate with Euler; ignore integrator option
    mj_Euler(m, d);
}
```

Simulation Loop

Contacts assumed soft, but not purely spring-damper-like

Relaxed complementarity $\rightarrow$ Allow penetration of rigid bodies

- Can solve as convex optimization, much faster
- “It’s a feature, not a bug.” Model compressibility, recover contact forces from kinematics.
- Choose parameter set to model the interaction you are looking for.





Thanks!

Feel free to reach out to me if you have questions!
bkh16@ic.ac.uk

For MuJoCo has easy to follow tutorials and documentation to getting started, as well as communities that can answer questions and solve problems:

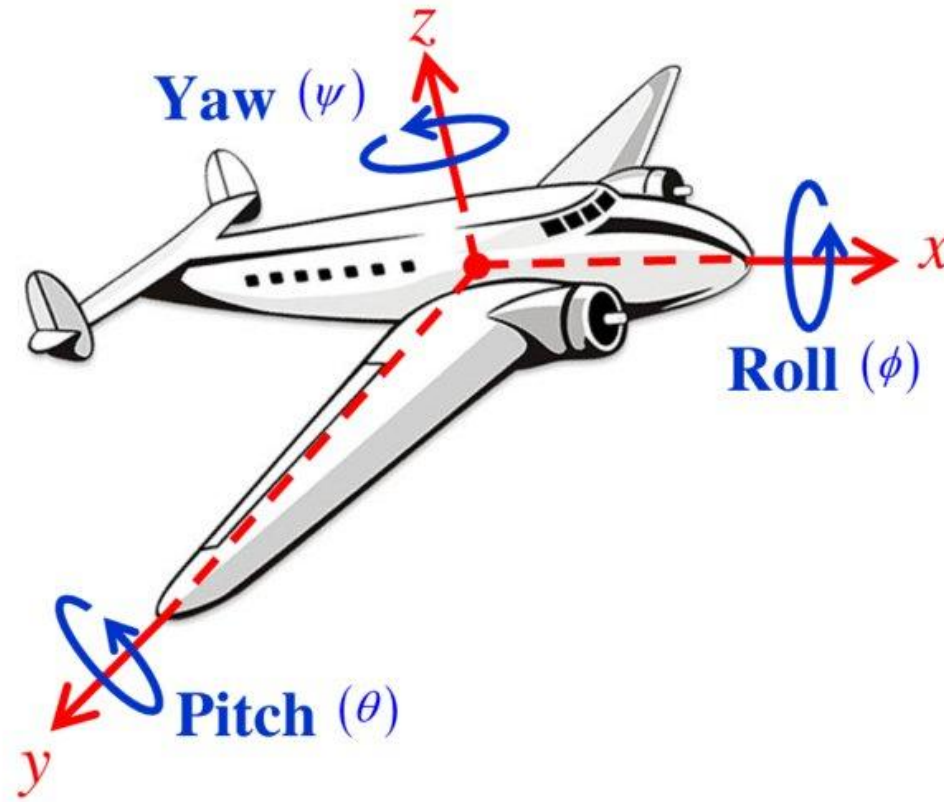
<https://github.com/google-deepmind/mujoco>

Recommended reading:

De Groote, F. and Falisse, A., 2021. Perspective on musculoskeletal modelling and predictive simulations of human movement to assess the neuromechanics of gait. *Proceedings of the Royal Society B*, 288(1946), p.20202432.

Representing 3 DoF rotations

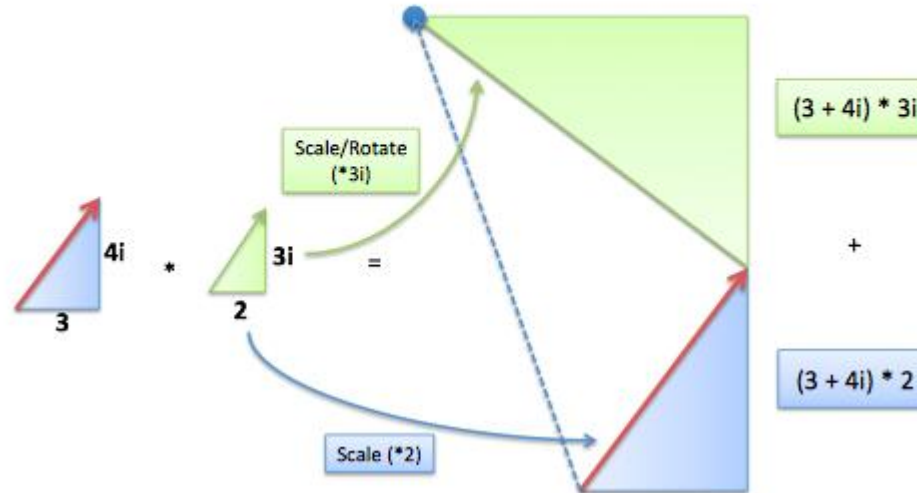
Euler angles are a simple and intuitive way, but suffer from a few key issues



Representing 3 DoF rotations

Unintuitively, you need more than three numbers to accurately represent 3 DoFs with shared center of rotation (e.g., ball joints or free joints).

Complex Multiplication



Representing 3 DoF rotations

Quaternions extend complex numbers to represent 3D rotations continuously, with familiar, multiplication-based notation.

4 numbers for position,
3 for velocity

Complex Multiplication

